

Otimização de balanceadores de carga de sistemas de distribuição de conteúdo por meio de monitoramento sistemático de memória

Pedro Faria Fernandes

Universidade do Estado de Santa Catarina

1 de dezembro de 2025

Sumário I

Introdução

Fundamentação Teórica

Proposta

Implementação

Experimentos e Resultados

Conclusão

Referências

Introdução

Contexto e Motivação

Evolução do Consumo de Vídeo

- ▶ Da **televisão** tradicional para a Internet;
- ▶ Consolidação do **vídeo sob demanda** como padrão;
- ▶ Crescimento **exponencial** do tráfego de vídeo na rede;

Distribuição de Conteúdo

- ▶ Replicação de conteúdo em múltiplos servidores;
- ▶ Redes de Distribuição de Conteúdo (**CDNs**) como solução central;
- ▶ Necessidade de **otimizar recursos** para manter a Qualidade de Experiência (QoE);

O Problema

- ▶ Sistemas de distribuição de conteúdo $\approx$ múltiplos servidores e um **balanceador de carga** central;
- ▶ Dispositivos de armazenamento persistente frequentemente são **gargalos** em cenários de *streaming* (consumo de conteúdo sob demanda);
- ▶ Uso intensivo de **memória principal** como *cache* alivia a memória persistente, mas torna a memória um recurso crítico;
- ▶ Muitas estratégias clássicas de balanceamento de carga ignoram o estado real de memória dos servidores;
- ▶ **Lacuna**: ausência de algoritmos que utilizem **exclusivamente** métricas de memória (consumo de memória principal e taxa de leitura de memória de armazenamento persistente) como parâmetro principal de decisão.

Objetivo

Objetivo do trabalho

- ▶ Avaliar uma estratégia de otimização da escolha de servidores em balanceadores de carga para sistemas de distribuição de conteúdo, baseada **exclusivamente** em métricas relevantes de memória;
- ▶ Implementar um algoritmo probabilístico de balanceamento de carga, apoiado em uma arquitetura de simulação completa;
- ▶ Realizar análise comparativa com algoritmos clássicos (*Round-Robin* e Seleção Aleatória) sob diferentes cenários de carga e heterogeneidade.

Fundamentação Teórica

Tópicos Centrais

- ▶ Balanceadores de carga em diferentes camadas (rede e aplicação) e balanceamento dinâmico [5, 7];
- ▶ Padrão **MPEG-DASH** para *streaming* adaptativo sobre HTTP [1, 9, 10];
- ▶ Arquitetura de **CDNs** e servidores de distribuição de conteúdo [8];
- ▶ Monitoramento de memória via APIs de sistemas operacionais (linux) [12, 11];
- ▶ Interpretação de dados de carga desatualizados [3, 4, 6];
- ▶ Virtualização de redes baseada em *containers* para simulação controlada [2].

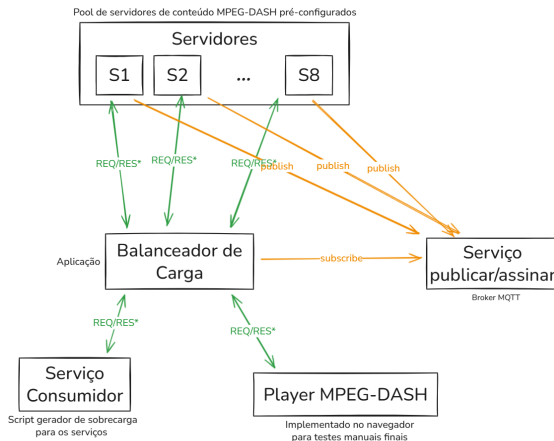
Proposta

Requisitos Tecnológicos

Componentes Principais

- ▶ **Servidores de Conteúdo:** 8 servidores MPEG-DASH, com capacidades variadas de memória principal e secundária (de acordo com os cenários de teste);
- ▶ **Balanceador de Carga:** Aplicação simples implementada do zero, responsável por aplicar os algoritmos de seleção de servidor;
- ▶ **Cliente Gerador de Carga:** Aplicação que simula múltiplos usuários MPEG-DASH concorrentes, iniciando múltiplas threads que fazem requisição de conteúdo para o balanceador de carga;
- ▶ **Cliente principal:** Serviço consumidor MPEG-DASH, que estará apontado para o balanceador de carga e possui o objetivo de coletar métricas e medir a QoE durante os cenários de teste;
- ▶ **Servidor MQTT:** *Broker* utilizado para transporte assíncrono de métricas entre servidores e balanceador.

Arquitetura Geral do Sistema



* Requisição e Resposta de conteúdo em áudio/vídeo

Arquitetura geral do sistema experimental (autor, 2025).

Estratégia de Balanceamento

- ▶ Definir o método de escolha de um servidor para cada nova requisição de vídeo;
- ▶ Algoritmo de natureza **probabilística**, baseado no **Basic Load Interpretation** [3];
- ▶ Probabilidade de escolha de cada servidor ajustada dinamicamente de acordo com:
 - ▶ Consumo de **memória principal**;
 - ▶ Taxa de leitura de **memória secundária**;
 - ▶ **Tempo de vida** das informações de carga.
- ▶ Servidores publicam periodicamente suas métricas de carga para o balanceador.

Cálculo da Probabilidade de Escolha do Servidor

A probabilidade (P_i) de uma requisição ser enviada para um servidor i é baseada no **Basic Load Interpretation**:

$$P_i = \frac{\frac{L_{tot} + arrive_T}{n} - L_i}{arrive_T} \quad (1)$$

P_i : Probabilidade de escolha do servidor i ;

n : Número total de servidores disponíveis;

L_i : Carga individual reportada pelo servidor i ;

L_{tot} : Somatório de todas as cargas reportadas ($\sum L_i$);

$arrive_T$: Carga esperada para os próximos T segundos, haja vista que T é o tempo médio de desatualização das informações de carga;

Pseudocódigo — Atualização da Distribuição

Algoritmo Atualização da distribuição de probabilidade

Gatilho: Nova informação de carga via MQTT.

Garante: Probabilidades P_i atualizadas

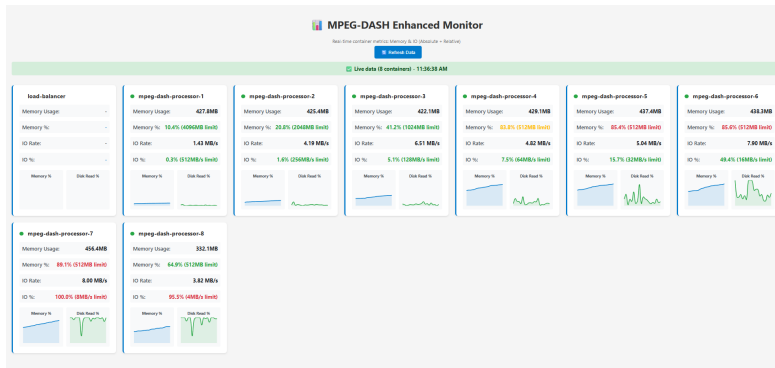
- 1: **Procedimento** ATUALIZARDISTRIBUICAO
 - 2: Atualizar métricas do servidor S_i
 - 3: Calcular L_i para cada servidor
 - 4: $L_{tot} \leftarrow \sum_{i=1}^n L_i$
 - 5: Calcular $arrive_T$
 - 6: Calcular P_i para cada servidor (Eq. 1)
 - 7: **Se** $n = 0$ **ou** $arrive_T \leq 0$ **então**
 - 8: $P_i \leftarrow 1/n$ para todos os servidores
-

Implementação

Ambiente Experimental

- ▶ Ambiente em **Windows + WSL2**, todos os componentes em contêineres **Docker**;
- ▶ Uso de **cgroups v2** para monitorar e limitar recursos (memória principal, leitura de memória secundária);
- ▶ 8 servidores MPEG-DASH (*ASP.NET Core*) com conteúdo exclusivo via *bind mounts*;
- ▶ Balanceador de carga em *Rust*, gerador de carga em *Python*;
- ▶ Métricas publicadas via **MQTT** (NanoMQ) periodicamente (tempo médio escolhido por cada servidor);
- ▶ Conteúdo MPEG-DASH com 6 resoluções (144p a 1080p) gerado via *ffmpeg*.

Painel de Monitoramento em Tempo Real



Painel de monitoramento em tempo real com métricas de consumo de memória principal e taxa de leitura de memória secundária.

Experimentos e Resultados

Cenários de Teste

Cenário	Usuários	Distribuição	Configuração dos Servidores
1	100	Heterogênea	S1: 4096MB / 1024MB/s; S2: 2048MB / 512MB/s; S3: 1024MB / 256MB/s; S4: 512MB / 128MB/s; S5: 256MB / 64MB/s; S6: 128MB / 32MB/s; S7: 64MB / 16MB/s; S8: 32MB / 8MB/s
2	100	Homogênea	Todos os servidores: 1024MB memória principal e 256MB/s de leitura de memória secundária
3	1000	Heterogênea	Mesma configuração do Cenário 1
4	1000	Homogênea	Mesma configuração do Cenário 2

Estratégias de Balanceamento

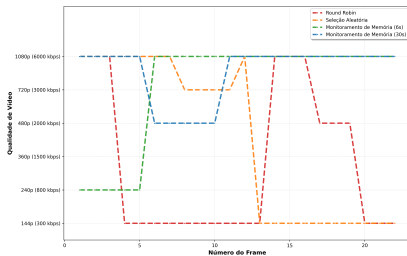
- ▶ **Round-Robin**: distribuição cíclica uniforme;
- ▶ **Seleção Aleatória**: servidor escolhido de forma uniforme e aleatória a cada requisição;
- ▶ **Monitoramento de Memória (6s)**: algoritmo proposto com atualização frequente de métricas;
- ▶ **Monitoramento de Memória (30s)**: algoritmo proposto com dados moderadamente desatualizados.

Métricas de Avaliação de QoE

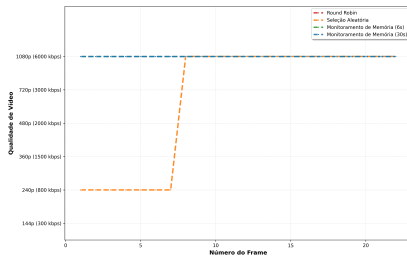
- ▶ **Latência Média** de resposta por requisição HTTP;
- ▶ **Qualidade** de vídeo ao longo da sessão (resoluções escolhidas pelo cliente);
- ▶ **Bitrate** médio por segmento;
- ▶ **Travamentos** (*stalls*): quantidade, duração total e maior duração.

Qualidade de Vídeo por Cenário

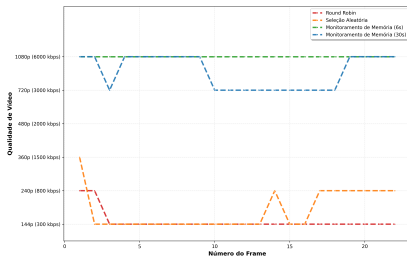
Qualidade de Vídeo por Frame - Cenário 1



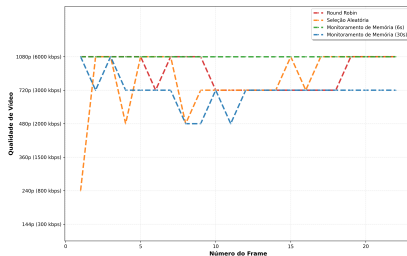
Qualidade de Vídeo por Frame - Cenário 2



Qualidade de Vídeo por Frame - Cenário 3

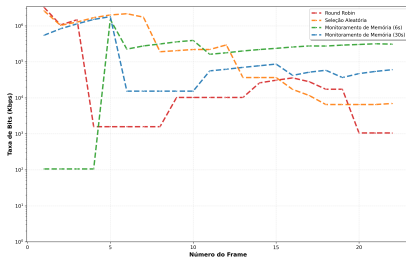


Qualidade de Vídeo por Frame - Cenário 4

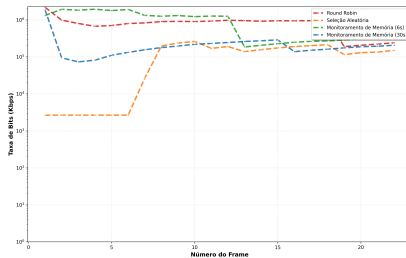


Bitrate por Cenário

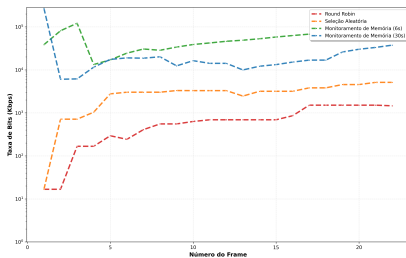
Taxa de Bits por Frame - Cenário 1



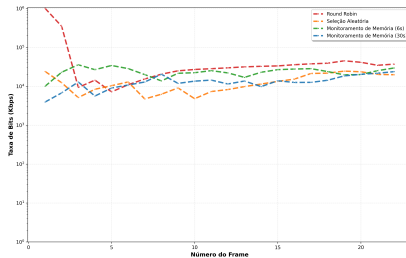
Taxa de Bits por Frame - Cenário 2



Taxa de Bits por Frame - Cenário 3

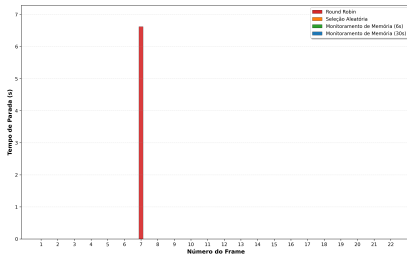


Taxa de Bits por Frame - Cenário 4

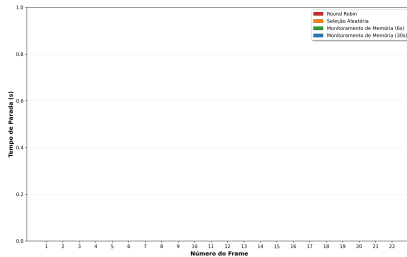


Stalls por Cenário

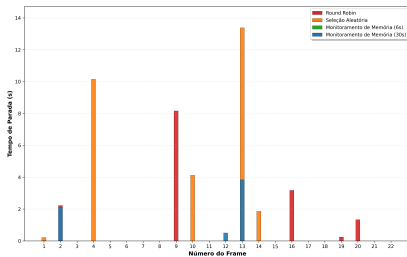
Duração de Parada por Frame - Cenário 1



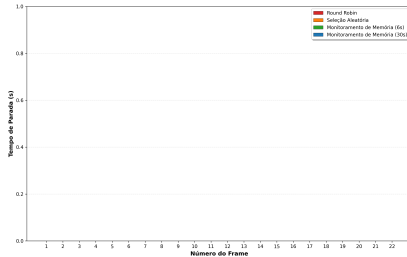
Duração de Parada por Frame - Cenário 2



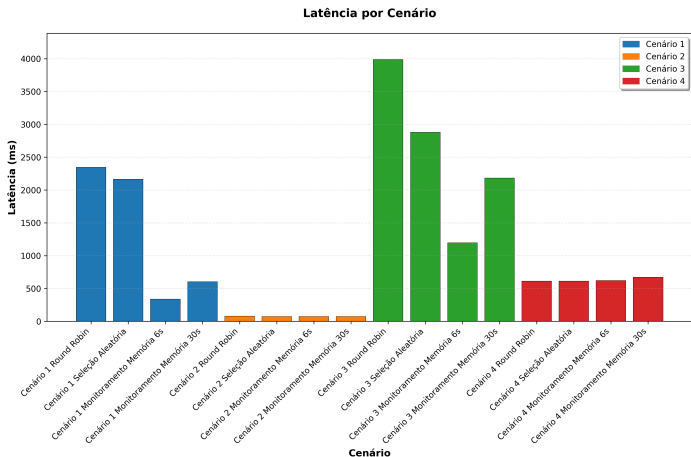
Duração de Parada por Frame - Cenário 3



Duração de Parada por Frame - Cenário 4



Latência Média por Cenário



Latência média de resposta por cenário e estratégia de balanceamento.

Conclusão

Fatores Determinantes do Desempenho

- ▶ **Heterogeneidade dos servidores:**

- ▶ Quanto maior a diferença de capacidade entre nós, maior o ganho do balanceamento inteligente;

- ▶ **Nível de concorrência:**

- ▶ Alta concorrência funciona como teste de estresse, expondo limitações de algoritmos simples;
- ▶ O algoritmo proposto mantém desempenho superior mesmo em cenários extremos (1000 usuários).

Contribuições do Trabalho

- ▶ Proposição de um **algoritmo de balanceamento** baseado em monitoramento de consumo de memória principal e taxa de leitura de memória secundária;
- ▶ Adaptação do conceito de **Load Interpretation** para métricas de recursos de baixo nível;
- ▶ Implementação completa e aberta com *C#*, *Rust*, *Python*, Docker e MQTT;
- ▶ Infraestrutura experimental reutilizável para avaliação de algoritmos de balanceamento em *streaming* MPEG-DASH;
- ▶ Demonstração de que heterogeneidade é fator crítico para o valor de estratégias inteligentes de balanceamento.

Limitações e Trabalhos Futuros

- ▶ Experimentos conduzidos em ambiente virtualizado único, sem latência geográfica real;
- ▶ Uso de um único algoritmo ABR (dash.js) e conjunto limitado de cenários de carga;
- ▶ Métricas focadas em memória principal e secundária, sem considerar rede e CPU como co-fatores;
- ▶ **Trabalhos futuros:**
 - ▶ Validação em ambientes distribuídos reais e CDNs de produção;
 - ▶ Extensão para múltiplas métricas (CPU, rede, latência geográfica);
 - ▶ Integração com SDN e *Content Steering* entre múltiplas CDNs;
 - ▶ Aplicação em cenários de *edge computing* e 5G.

Reprodutibilidade e Considerações Finais

- ▶ Todo o código, scripts e dados estão disponíveis em:
 - ▶ <https://github.com/peedrofernandes/memory-oriented-load-balancer.git>
- ▶ A infraestrutura experimental permite reproduzir todos os 16 cenários de teste;
- ▶ Resultados demonstram que **monitorar e interpretar adequadamente o uso de recursos** é essencial para QoE em CDNs heterogêneas;
- ▶ Estratégias de balanceamento conscientes de memória representam um caminho promissor para a próxima geração de sistemas de distribuição de conteúdo.

Referências

Referências I

- [1] Ali Begen, Tankut Akgul, and Mark Baugher. Watching video over the web: Part 1: Streaming protocols. *IEEE Internet Computing*, 15(2), 2011.
- [2] N.M. Mosharaf Kabir Chowdhury and Raouf Boutaba. A survey of network virtualization. *Computer Networks*, 54(5), 2010.
- [3] M. Dahlin. Interpreting stale load information. *IEEE Transactions on Parallel and Distributed Systems*, 11(10), 2000.
- [4] Derek L. Eager, Edward D. Lazowska, and John Zahorjan. Adaptive load sharing in homogeneous distributed systems. *IEEE Transactions on Software Engineering*, SE-12(5), 1986.
- [5] Kimberly Jane. Scalability issues in database systems: Strategies for scaling databases horizontally and vertically. 03 2025.
- [6] R. Mirchandaney, D. Towsley, and J.A. Stankovic. Analysis of the effects of delays on load sharing. *IEEE Transactions on Computers*, 38(11), 1989.

Referências II

- [7] A. Pasumpon Pandian, Xavier Fernando, and Wang Haoxiang, editors. *Computer Networks, Big Data and IoT: Proceedings of ICCBI 2021*, volume 117 of *Lecture Notes on Data Engineering and Communications Technologies*. Springer Nature Singapore, 2022.
- [8] Sam Preston. Finding the best servers to answer queries—edge dns and anycast, March 2021. Akamai Blog.
- [9] Iraj Sodagar. The mpeg-dash standard for multimedia streaming over the internet. *IEEE MultiMedia*, 18(4), 2011.
- [10] Thomas Stockhammer. Dynamic adaptive streaming over http: Standards and design principles. 02 2011.
- [11] The Linux Kernel Developers. I/o statistics fields for block devices, January 2011. Descreve os campos fornecidos pelo arquivo `/proc/diskstats`.
- [12] The Linux Kernel Developers. The `/proc` filesystem, March 2020. Documentação do kernel sobre a estrutura e os arquivos do sistema de arquivos `/proc`, incluindo `/proc/meminfo`.